Technical report - ClearDesk

- Track: Track 2B - Own Project

- Event: Online Hack Apertus, October 2026

- Author: Mohammed Naser (m7mdd77), solo

- Demo: local 70-second playback of actual response artifacts; not a live screen recording; not hosted or included in the source publication

- Submission: not submitted

1. Summary

ClearDesk uses local Apertus to select IDs and verbatim passages from synthetic IT procedures. Real generation has run under the user's local-only AUP approval, including genuine root GNU Make and Docker inference. The validator rejects fabricated quotes, unknown IDs, extra fields and multiple citations. Missing evidence produces abstention, but the baseline model failed to abstain twice. The application takes no actions. Reliable relevance beyond the development fixtures remains unverified.

2. Architecture

Question -> local Apertus 1.5 chat completion -> JSON/schema check -> exact quote check -> cited evidence or abstention. Source URLs and titles come from the corpus rather than the model. The output does not contain an unconstrained model-authored answer.

Target architecture

On-premise target: Node CLI and llama.cpp on infrastructure controlled by the operator. Docker Compose uses an internal network without published ports. Build time downloads runtime dependencies, not weights. The separately acquired GGUF is mounted read-only and hashed by an asset-check container before startup; it is not redistributed in the image. A project-root Makefile invokes the Node container runner. Actual runtime inspection confirmed RW=false, no host ports and Internal=true. No external-connectivity probe was performed. Public AI APIs are not used.

3. Use of Apertus

Model candidate: Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF, revision 248ec68a63e219e3f061e4c946fc8a20d63b9975. This is a community text-only quantization derived from Andreas Martin's conversion of swiss-ai/Apertus-v1.5-8B, not an official multimodal checkpoint. The official FAQ permits community models for Track 2. See docs/provenance.md for architecture changes and rights.

Serving: llama.cpp b11459, commit f498f864fbc0472004ee1c3616c1188c68eb157f. Temperature 0, seed 7, output limit 500 tokens. System prompt and request contract are in src/assistant.mjs. The model performs semantic selection across English, German and French questions; exact source matching is not substituted for inference.

Coding assistance: Codex generated and edited the implementation and synthetic fixtures. The author must review the final submission; no personal experience or reflection is invented. No other model serves as an automatic evaluation judge.

4. Data

Four fictional IT procedure records and seven synthetic evaluation questions created on October 7 for this project. No human subject records, secrets, real institutional policies or personal data. example.invalid references are fixture identifiers, not live integrations. Data directory is far below the 100 MB limit. The author confirmed rights and approved CDLA-Permissive-2.0 for this synthetic dataset.

5. Evaluation

Contract suite: 22/22 tests passed with Node's built-in test runner on October 7. These cover quotation/schema gates, identified transport doubles, real fixture hashing, single-source enforcement and unapproved-run refusal. These are separate from live inference. Seven actual cases cover routine retrieval, multilingual questions, an unrelated question and an instruction-injection question. Success requires the expected source ID and relevant quote fragments; unknown/injection cases require the model's empty response. Validation-induced abstention is not scored as successful model reasoning.

Measured baseline: 5/7 expected-source checks, with irrelevant extra citations and two failed abstentions. Preserved in docs/model-evaluation-vulkan.json. Revised one-source prompt/schema: GPU 7/7 in docs/model-evaluation-vulkan-1791385931354.json; CPU-only 7/7 in docs/model-evaluation-cpu-1791386027334.json. Both returned exactly one expected source for all five answerable questions and empty evidence for unknown/injection cases. Request-time totals: GPU 34,733 ms; CPU 106,476 ms, excluding startup/hash checks and including within-run prompt caching. These are small synthetic runs on one workstation, not general accuracy or latency benchmarks. The prompt was revised after seeing baseline failures, so this is development-set evidence, not held-out validation. Always-abstain scores 2/7 by fixture definition, not a separately executed model baseline.

6. Limitations

Docker development evaluation passed 7/7 in docs/model-evaluation-endpoint-1791469094778.json: total API request time 86,238 ms, excluding startup/hash checks and including prompt caching after a smoke request. The container smoke returned the exact IT-101 lost-laptop passage in 17,939 ms (first root run: 19,798 ms). These figures are measured workloads, not a controlled CPU-versus-GPU performance comparison.

Small synthetic evaluation, no production deployment or user study. Exact quotations can be irrelevant or incomplete, as observed. No certified injection resistance. Model is a community conversion without official logit-parity guarantees. Docker and root container inference were tested on the existing workstation, not a fresh machine or clean public checkout. Corpus is deliberately capped. User explicitly approved AUP obligations for local synthetic inference only. Source/report/data publication is approved; weight redistribution, third-party demo hosting and final contest acceptance remain outside this approval.

7. Reproducibility

npm test passed 22/22. The local runner hashes the 5,059,027,136-byte GGUF; SHA256 a037df8d87ff6caacee794ee85f55342f2152e0d359b7389033300c3bee5f299 was verified in PowerShell, Node and the Docker asset-check container. Revised GPU logs confirm 33/33 layers on Vulkan0 / RTX 5060, 4528.69 MiB model and 512 MiB KV buffers. CPU-only uses --device none, --no-op-offload and --no-kv-offload: log confirms 0/33 layers, CPU weights/KV/86.02 MiB compute buffer, no GPU compute buffer. Older zero-layer-offload smoke was hybrid and is not pure CPU proof. Hardware: RTX 5060, i5-14400F, 31.8 GB RAM. Raw response evidence and logs are preserved. The Docker root runner builds, waits for model health, inspects runtime state, requests real inference and removes its own containers/network on completion. Set CLEARDESK_EVALUATE=1 to also save seven timestamped development-case results.

8. Next steps

Publish the approved source/report/data through the public template-derived repository and check the public checkout. Demo hosting and final contest acceptance require separate approval. Docker Desktop was opened by the user; no host repair or global GNU Make installation was performed. The existing local 70-second demo remains labeled actual-output playback, not live screen capture or Docker proof. Full decode verified 700 frames at 10 fps, 1280x720. Beyond the event: held-out responsibly sourced evaluation and relevance/entailment review.

October 8 update superseding the earlier engine-check failure: server 29.6.2 is available through authorized Docker commands. Genuine project-local GNU Make 4.4.1 executed root make test (22/22) and root make run twice (exit 0). All three images built, including 22/22 Linux container tests. The asset check confirmed the existing model hash. Live mount/port/network checks and owned-container cleanup succeeded. Screened evidence is in docs/docker-verification.json. Build contexts exclude model weights, reports, video and credentials.

License

Copyright 2026 Mohammed Naser. This report is licensed under CC-BY-4.0 with explicit author approval and rights confirmation. Full text: licenses/CC-BY-4.0.txt. Code uses Apache-2.0; synthetic data uses CDLA-Permissive-2.0. Upstream template notices are retained. No model weights or final contest submission warranties are included in this grant.

References

- https://hackapertus.ch/faq
- https://github.com/HackApertus/project-template/tree/main/track_2b
- https://hackapertus.ch/terms-and-conditions
- https://huggingface.co/Colby/apertus-v1.5-8b-text-Q4_K_M-GGUF
- https://huggingface.co/andreasmartin/apertus-v1.5-8b-text
- https://github.com/ggml-org/llama.cpp/releases/tag/b11459